

LoongKEM: Algorithm Specifications And Supporting Documentation

Chengdong Tao

June 27, 2026

Contents

1	Introduction	3
2	Written specification	6
2.1	Preliminaries and notation	6
2.2	Loong public key encryption	15
2.2.1	Keypair generation	15
2.2.2	Encryption	16
2.2.3	Decryption	17
2.3	LoongKEM	20
2.4	Parameters	21
3	Design principle	22
3.1	Semi-structured LWE problem	23
3.2	Error distribution	24
4	Performance analysis	24
4.1	Performance	24
4.2	Known answer test values	25
5	Security analysis	26
5.1	Security proof	26
5.2	Complexity of the BKZ algorithm	27
5.3	Practical security evaluation of the LoongKEM	27
5.3.1	Primal Attack	28
5.3.2	Dual Attack	28
5.3.3	Attack Complexity of the LoongKEM	28
6	Advantages and limitations	29

1 Introduction

This submission presents the basic principle and implementation of the Loong key encapsulation mechanism (LoongKEM for short). The LoongKEM is a family of lattice-based cryptographic algorithms whose security base on the hardness of the semi-structured Learning With Errors problem (SLWE for short).

Early studies on lattice-based cryptography date back to the 1990s. In 1996, Ajtai proved that the hardness of lattice problems such as the Small Integer Solution (SIS) problem is equivalent between the average case and the worst case [1]. This result implies that lattice problems are particularly suitable for constructing public-key cryptographic primitives. Many mathematical problems underlying public-key cryptosystems lack such an equivalence proof, including the integer factorization problem, the discrete logarithm problem, the decoding problem of random linear codes, and the problem of solving multivariate polynomial systems. In the same work, Ajtai also constructed a hash function based on SIS, which pioneered the field of lattice-based cryptography.

In 1996, Hoffstein, Pipher, and Silverman proposed the NTRU public-key encryption scheme [2]. NTRU can be viewed as a lattice-based cryptosystem with fast encryption and decryption, making it attractive for practical applications. However, the original NTRU scheme does not come with a formal security proof, and its security is only evaluated against known attacks.

In 1997, Ajtai constructed a public-key encryption scheme based on the unique Shortest Vector Problem (uSVP), a well-known hard lattice problem [3]. The security of this scheme can be reduced to the hardness of uSVP, meaning that any polynomial-time adversary breaking the scheme implies a polynomial-time algorithm for solving uSVP. Unfortunately, this scheme suffers from low encryption and decryption efficiency, which limits its real-world deployment.

In 2005, Regev introduced the Learning With Errors (LWE) problem and built a public-key encryption scheme based on LWE [4], which constitutes a landmark breakthrough in lattice cryptography. The public key and ciphertext sizes of Regev’s scheme are both $O(n^2)$, where n denotes the dimension of the public-key matrix.

To reduce the size of public keys and ciphertexts, in 2010, Lyubashevsky, Peikert, and Regev proposed the Ring-LWE (RLWE) problem and constructed corresponding public-key encryption schemes [5]. The public key and ciphertext sizes of RLWE-based schemes are $\mathcal{O}(n)$.

To further compress ciphertexts, Ding et al. essentially proposed a ciphertext compression technique and applied it to design lattice-based key exchange protocols [6]. Peikert extended this technique and applied it to RLWE-based encryption schemes, achieving shorter ciphertexts [7, 8].

The RLWE problem embodies the algebraic structure of polynomial rings. Although no efficient algebraic attacks against RLWE-based cryptosystems are known to date, such schemes still bear potential risks from algebraic attacks. Accordingly, in 2015, [9] introduced the Module-LWE (MLWE) problem. MLWE preserves partial algebraic structure from RLWE to achieve compact public keys and ciphertexts as well as high efficiency, while avoiding the strong alge-

braic assumptions of RLWE. MLWE thus provides a favorable trade-off between security and efficiency and is widely adopted in practical cryptosystems. Nevertheless, existing MLWE-based schemes still retain relatively strong algebraic structures, such as the CRYSTALS-Kyber key encapsulation mechanism [10] and the CRYSTALS-Dilithium signature scheme [11].

To alleviate the security concerns caused by strong algebraic structures, this submission introduces a key encapsulation mechanism based on the semi-structured LWE problem. The LoongKEM is a family of IND-CCA2 secure constructions. We first build a CPA-secure public-key encryption scheme, then transform it into an IND-CCA2 secure key encapsulation mechanism via the Fujisaki-Okamoto transformation.

For clarity, we briefly describe the CPA-secure Loong public-key encryption scheme. For simplicity, we omit the compression and decompression steps for public keys and ciphertexts. In the original LWE-based encryption schemes, the public key is typically a generic matrix without any algebraic structure. In MLWE-based and RLWE-based encryption schemes, the public key is usually a matrix with algebraic structure isomorphic to some polynomial ring, such as $\mathbb{Z}_q[X]/X^N + 1$ or $\mathbb{Z}_q[X]/X^N - 1$. In the Loong public-key encryption scheme, the public-key matrices $(\mathbf{A}, \mathbf{B})$ are semi-structured. Specifically,

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix},$$

where $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3, \mathbf{B}_1$ are block anti-circulant matrices. Each block can be viewed as a polynomial in the ring $\mathbb{Z}_q[X]/X^N + 1$. The matrices $\mathbf{A}_4$ and $\mathbf{B}_2$ are generic matrices without algebraic structure.

Let N , k_1 and k_2 be positive integers, and let q be a prime with $q > 2$. The key generation of the Loong public-key encryption scheme proceeds as follows:

1. Randomly generate the matrix $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}$, where the submatrices $\mathbf{A}_1 \in \mathbb{Z}_q^{k_1 N \times k_1 N}$, $\mathbf{A}_2 \in \mathbb{Z}_q^{k_1 N \times k_2 N}$, $\mathbf{A}_3 \in \mathbb{Z}_q^{k_2 N \times k_1 N}$ are block anti-circulant matrices, with each block being an anti-circulant matrix. The matrix $\mathbf{A}_4 \in \mathbb{Z}_q^{k_2 N \times k_2 N}$ is a generic matrix without algebraic structure.
2. Sample noise matrices $\begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}$ and $\begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix}$ from the centered binomial distribution, where $\mathbf{S}_1$ and $\mathbf{E}_1$ are $k_1 N \times N$ block anti-circulant matrices, $\mathbf{S}_2$ is a $k_2 N \times N$ block anti-circulant matrices. $\mathbf{E}_2$ is a $k_2 N \times N$ generic matrix without algebraic structure.
3. Compute $\mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix}$, where $\mathbf{B}_1 \in \mathbb{Z}_q^{k_1 N \times N}$ is a block anti-circulant matrix and $\mathbf{B}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ is a generic matrix without algebraic structure.
4. The public key consists of $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix}$, and the

private key is $\mathbf{S} = \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}$. The matrix $\mathbf{A}$ can be generated from a random seed, so only the seed needs to be stored. Since $\mathbf{B}_1$ is block anti-circulant, only the first row of each block needs to be stored. Since $\mathbf{B}_2$ is a generic matrix, all its entries must be stored.

Let $\mathbf{m} \in \{0, 1\}^{N^2}$ be the plaintext bits. We first encode the plaintext into a binary $N \times N$ matrix $\mathbf{M}$. Let $\Delta = 2^{-1} \bmod q$. The encryption of the Loong public-key encryption scheme is as follows:

1. Sample noise matrices $(\mathbf{R}_1 \ \mathbf{R}_2)$, $(\mathbf{E}_3 \ \mathbf{E}_4)$ and $\mathbf{E}_5$ from the centered binomial distribution, where $\mathbf{R}_1$ is a $N \times k_1 N$ block anti-circulant matrix, $\mathbf{R}_2$ is a $N \times k_2 N$ block anti-circulant matrix, $\mathbf{E}_3$ is a $N \times k_1 N$ block anti-circulant matrix, $\mathbf{E}_4$ is a $N \times k_2 N$ generic matrix without algebraic structure, $\mathbf{E}_5$ is a $N \times N$ generic matrix without algebraic structure.
2. Compute $(\mathbf{C}_1 \ \mathbf{C}_2) = (\mathbf{R}_1 \ \mathbf{R}_2) \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} + (\mathbf{E}_3 \ \mathbf{E}_4)$, where $\mathbf{C}_1 \in Z_q^{N \times k_1 N}$ is a block anti-circulant matrix and $\mathbf{C}_2 \in Z_q^{N \times k_2 N}$ is a generic matrix without algebraic structure.
3. Compute $\mathbf{C}_3 = (\mathbf{R}_1 \ \mathbf{R}_2) \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} + \mathbf{E}_5 + \Delta \cdot \mathbf{M}$, where $\mathbf{C}_3 \in Z_q^{N \times N}$ is a generic matrix without algebraic structure.
4. The ciphertext is $\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3$. Since $\mathbf{C}_1$ is block anti-circulant, only the first row of each block needs to be stored. Since $\mathbf{C}_2$ and $\mathbf{C}_3$ are generic matrices, all their entries must be stored. The total number of entries to be stored is $k_1 N + k_2 N^2 + N^2$.

The decryption of the Loong public-key encryption scheme is as follows:

1. Compute $\mathbf{U} = 2(\mathbf{C}_3 - \mathbf{C}_1 \mathbf{S}_1 - \mathbf{C}_2 \mathbf{S}_2) \bmod \pm q$.
2. Treat $\mathbf{U}$ as an integer matrix and compute $\mathbf{M} = \mathbf{U} \bmod 2$, The matrix $\mathbf{M}$ is the plaintext.

In conventional LWE-based encryption schemes, the final step of decryption usually compares the ciphertext with $q/4$: if larger than $q/4$, the corresponding plaintext bit is 1; otherwise, it is 0. In the Loong encryption scheme, we only perform a modular operation to recover the plaintext. This is similar to the decryption of NTRU and improves decryption efficiency. We summarize the Loong public-key encryption scheme in Table 1.

The rest of this paper is organized as follows. Section 2 describes the Loong key encapsulation mechanism and its parameters in detail. Section 3 presents the design rationale of the LoongKEM. Section 4 introduces the implementation and performance of the LoongKEM. Section 5 analyzes the security of the LoongKEM. Section 6 discusses the advantages and disadvantages of the LoongKEM.

KeyGen:

- 1: $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \leftarrow \mathbb{Z}_q^{kN \times kN}, \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} \leftarrow \psi_\eta, \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix} \leftarrow \psi_\eta$
- 2: $\mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} \leftarrow \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix}$
- 3: **return** $\text{pk} = (\mathbf{A}, \mathbf{B}), \text{sk} = \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}$

Encrypt(pk, M):

- 1: $(\mathbf{R}_1 \ \mathbf{R}_2) \leftarrow \psi_\eta, (\mathbf{E}_3 \ \mathbf{E}_4) \leftarrow \psi_\eta, \mathbf{E}_5 \leftarrow \psi_\eta$
- 2: $(\mathbf{C}_1 \ \mathbf{C}_2) \leftarrow (\mathbf{R}_1 \ \mathbf{R}_2) \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} + (\mathbf{E}_3 \ \mathbf{E}_4)$
- 3: $\mathbf{C}_3 \leftarrow (\mathbf{R}_1 \ \mathbf{R}_2) \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} + \mathbf{E}_5 + \Delta \cdot \mathbf{M}$
- 4: **return** $(\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3)$

Decrypt(sk, C₁, C₂, C₃):

- 1: $\mathbf{M} \leftarrow (2(\mathbf{C}_3 - \mathbf{C}_1 \mathbf{S}_1 - \mathbf{C}_2 \mathbf{S}_2) \bmod^{\pm} q) \bmod 2$
 - 2: **return** $\mathbf{M}$
-

Table 1: CPA-secure Loong public-key encryption scheme

2 Written specification

2.1 Preliminaries and notation

Integer modular arithmetic. Let $\mathbb{Z}$ be the ring of integers, q be an odd prime, and $\mathbb{Z}_q$ be the quotient ring $\mathbb{Z}/q\mathbb{Z}$. For an integer a , the modular operation $r = a \bmod q$ means $r \in \{0, 1, \dots, q-1\}$ and modular operation $r = a \bmod^{\pm} q$ means $r \in \{-\frac{q-1}{2}, \dots, -1, 0, 1, \dots, \frac{q-1}{2}\}$. We denote $d_q = \lceil \log_2(q) \rceil$.

Norms. Let $a \in \mathbb{Z}_q$ and $\mathbf{a} = (a_0, a_1, \dots, a_{n-1}) \in \mathbb{Z}_q^n$. We define

$$\|a\|_{\infty} = |a \bmod^{\pm} q|, \quad \|\mathbf{a}\|_{\infty} = \max_{0 \leq i \leq n-1} \|a_i\|_{\infty}.$$

$$\|\mathbf{a}\|_2 = \left(\|a_0\|_{\infty}^2 + \dots + \|a_{n-1}\|_{\infty}^2 \right)^{1/2}.$$

Compression and Decompression. Let d be a positive even integer and $x \in \{0, 1, \dots, q-1\}$, We define the compression function as follows:

$$\text{Compress}(x, d) = \left\lfloor \frac{x}{d} \right\rfloor.$$

Let $y = \text{Compress}(x, d)$, We define the decompression function as follows:

$$\text{Decompress}(y, d) = \min \left(y \cdot d + \frac{d}{2}, q-1 \right).$$

When d is a power of two, the compression and decompression of elements in $\mathbb{Z}_q$ can be implemented using only shift and addition operations, without division or modular operations.

Let $\tilde{x} = \text{Decompress}(\text{Compress}(x, d), d)$. For any $x \in \mathbb{Z}_q$, it is easy to show that $|x - \tilde{x}| \leq \frac{d}{2}$. Indeed, by the definition of the floor function, there exists an integer $r \in [0, d)$ such that $x = yd + r$. We prove this by considering two cases. If $yd + \frac{d}{2} \leq q-1$, then $|x - \tilde{x}| = |r - \frac{d}{2}| \leq \frac{d}{2}$. If $yd + \frac{d}{2} > q-1$, then $\tilde{x} = q-1$. On the other hand, $x \geq yd > (q-1) - \frac{d}{2}$ (Otherwise $yd + \frac{d}{2} \leq q-1$). Thus $|x - \tilde{x}| = (q-1) - x < \frac{d}{2}$.

Let $\mathbf{a} \in \mathbb{Z}_q^n$ and $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$, $\text{Compress}(\mathbf{a}, d)$ and $\text{Compress}(\mathbf{A}, d)$ denote the compression operation applied to each entry of the vector $\mathbf{a}$ and the matrix $\mathbf{A}$, respectively. $\text{Decompress}(\mathbf{a}, d)$ and $\text{Decompress}(\mathbf{A}, d)$ denote the decompression operation applied to each entry of the vector $\mathbf{a}$ and the matrix $\mathbf{A}$, respectively.

Bit String and Byte Array. Let the set $\mathbb{B} = \{0, 1, 2, \dots, 255\}$ and t be a positive integer. Algorithm 1 converts a bit array $\mathbf{b}$ into a byte array $\mathbf{bytes}$. Given a bit string $b_0b_1b_2b_3b_4b_5b_6b_7$, the corresponding byte is $b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_7 \cdot 2^7$, i.e., we adopt the least significant bit (LSB) first order. For example, the bit string 11111011 corresponds to the byte $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^6 + 2^7 = 223$.

Algorithm 1 BitsToBytes($\mathbf{b}$): Convert a bit array to a byte array.

Input: $\mathbf{b} \in \{0, 1\}^{8t}$.

Output: $\mathbf{bytes} \in \mathbb{B}^t$.

```

1:  $\mathbf{bytes} \leftarrow (0, \dots, 0)$ 
2: for ( $i = 0; i < t; i++$ ) do
3:    $\mathbf{bytes}[i] \leftarrow \sum_{j=0}^7 \mathbf{b}[8i+j] \cdot 2^j$ 
4: end for
5: return  $\mathbf{bytes}$ 
```

Algorithm 2 converts a byte array $\mathbf{bytes}$ into a bit array $\mathbf{b}$. Let an element of bytes be $b_0 + b_1 \cdot 2 + \dots + b_7 \cdot 2^7$, then its corresponding bit string is $b_0b_1 \dots b_7$.

Algorithm 2 BytesToBits(**bytes**): Convert a byte array to a bit array.

Input: **bytes** $\in \mathbb{B}^t$.

Output: **b** $\in \{0, 1\}^{8t}$.

```

1: c  $\leftarrow$  bytes
2: for ( $i = 0; i < t; i++$ ) do
3:   for ( $j = 0; j < 8; j++$ ) do
4:     b[ $8i + j$ ]  $\leftarrow$  c[ $i$ ] mod 2
5:     c[ $i$ ]  $\leftarrow$   $\lfloor \mathbf{c}[i]/2 \rfloor$ 
6:   end for
7: end for
8: return b

```

Integer and byte array. Let d be a positive integer. Algorithm 3 converts an integer array (where each element lies in the interval $[0, 2^d - 1]$ and the array length is n) into a byte array. In the LoongKEM, n is a multiple of 8. For an element $b_0 + b_1 \cdot 2 + \dots + b_{d-1} \cdot 2^{d-1}$ in the integer array **c**, its corresponding bit string is $b_0 b_1 \dots b_{d-1}$. The function **ByteEncode** first transforms the integer array $c_0, c_1, \dots, c_{n-1}$ of length n into a bit string $b_0, b_1, \dots, b_{nd-1}$, then invokes the **BitsToBytes** function to convert this bit string into a byte array.

Algorithm 3 ByteEncode(**c**, n , d): Convert an integer array to a byte array.

Input: **c** $\in \mathbb{Z}_{2^d}^n$, n and d .

Output: **bytes** $\in \mathbb{B}^{nd/8}$.

```

1: b  $\leftarrow (0, 0, \dots, 0)$  ▷ The length of b is  $nd$ .
2: for ( $i = 0; i < n; i++$ ) do
3:    $a \leftarrow \mathbf{c}_i$ 
4:   for ( $j = 0; j < d; j++$ ) do
5:     b[ $i \cdot d + j$ ]  $\leftarrow a$  mod 2
6:      $a \leftarrow (a - \mathbf{b}[i \cdot d + j])/2$ 
7:   end for
8: end for
   bytes  $\leftarrow$  BitsToBytes(b)
9: return bytes

```

Algorithm 4 converts a byte array to an integer array. First, the **BytesToBits** function is used to convert the byte array into a bit string in least significant bit (LSB) first order. The bit string is then converted into an integer array.

Algorithm 4 ByteDecode(**bytes**, n , d): Convert a byte array to an integer array.

Input: **bytes** $\in \mathbb{B}^{nd/8}$, n and d .

Output: **c** $\in \mathbb{Z}_{2^d}^n$.

```

1: b  $\leftarrow$  BytesToBits(bytes)
2: for ( $i = 0; i < n; i++$ ) do
3:   c[ $i$ ]  $\leftarrow \sum_{j=0}^{d-1} \mathbf{b}[i \cdot d + j] \cdot 2^j \pmod{2^d}$ 
4: end for
5: return c

```

Polynomial ring and anti-circulant matrix. Let N be a positive integer. We define the polynomial rings $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. The coefficients of polynomials in $\mathcal{R}$ and $\mathcal{R}_q$ are defined over $\mathbb{Z}$ and $\mathbb{Z}_q$, respectively. We use bold lowercase letters to denote elements of $\mathcal{R}_q$. For any $\mathbf{c}(X) = c_0 + c_1X + \dots + c_{N-1}X^{N-1} \in \mathcal{R}_q$, we usually write it as $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$. Unless ambiguity arises, a bold lowercase letter sometimes denotes a polynomial and sometimes denotes its coefficient vector. Let $\mathbf{C}$ be the matrix corresponding to $\mathbf{c}(X)$, where the i -th row ($i = 0, 1, \dots, N-1$) of $\mathbf{C}$ consists of the coefficients of $X^i \mathbf{c}(X) \pmod{X^N + 1}$. Specifically,

$$\mathbf{C} = \begin{pmatrix} c_0 & c_1 & c_2 & \cdots & c_{N-2} & c_{N-1} \\ -c_{N-1} & c_0 & c_1 & \cdots & c_{N-3} & c_{N-2} \\ -c_{N-2} & -c_{N-1} & c_0 & \cdots & c_{N-4} & c_{N-3} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -c_1 & -c_2 & -c_3 & \cdots & -c_{N-1} & c_0 \end{pmatrix}.$$

We call a matrix of the form $\mathbf{C}$ an anti-circulant matrix. Let the polynomial matrix

$$\mathbf{b} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,k-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{k-1,0} & \mathbf{b}_{k-1,1} & \cdots & \mathbf{b}_{k-1,k-1} \end{pmatrix},$$

where $\mathbf{b}_{ij} \in R_q$ ($0 \leq i \leq k-1, 0 \leq j \leq k-1$). We define the matrix corresponding to the polynomial matrix $\mathbf{b}$ as

$$\mathbf{B} = \begin{pmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} & \cdots & \mathbf{B}_{0,k-1} \\ \mathbf{B}_{10} & \mathbf{B}_{11} & \cdots & \mathbf{B}_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{B}_{k-1,0} & \mathbf{B}_{k-1,1} & \cdots & \mathbf{B}_{k-1,k-1} \end{pmatrix},$$

where $\mathbf{B}_{ij}$ is the anti-circulant matrix corresponding to the polynomial $\mathbf{b}_{ij}$. We call a matrix of the form $\mathbf{B}$ a block anti-circulant matrix. Each sub-block $\mathbf{B}_{ij}$ ($0 \leq i \leq k-1, 0 \leq j \leq k-1$) of $\mathbf{B}$ is an anti-circulant matrix.

Algorithm 5 converts a polynomial into an anti-circulant matrix. The input of the algorithm is the coefficient vector of the polynomial, and the output is the corresponding anti-circulant matrix of the polynomial. Algorithm 6 converts a polynomial matrix into a block anti-circulant matrix. The input of the algorithm is a polynomial matrix, and the output is a block anti-circulant matrix.

Algorithm 5 PolyToMat(**c**)

Input: $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$.

Output: $\mathbf{C}$.

```

1: for  $i = 0$  to  $N - 1$  do
2:   for  $j = 0$  to  $N - 1$  do
3:     if  $j \geq i$  then
4:        $\mathbf{C}[i][j] \leftarrow c[j - i]$ 
5:     else
6:        $\mathbf{C}[i][j] \leftarrow -c[N + j - i]$ 
7:     end if
8:   end for
9: end for
10: return  $\mathbf{C}$ 

```

Algorithm 6 PolysToBlockMat($\mathbf{A}, k_1, k_2$)

Input: k_1, k_2 and $k_1 \times k_2$ polynomial matrix $\mathbf{A}$, where each entry $\mathbf{A}[i][j]$ is a polynomial coefficient vector of length N .

Output: Block anti-circulant matrix $\mathbf{B} \in \mathbb{Z}_q^{k_1 N \times k_2 N}$

```

for  $i = 0$  to  $k_1 - 1$  do
  for  $j = 0$  to  $k_2 - 1$  do
     $\mathbf{T} \leftarrow \text{PolyToMat}(\mathbf{A}[i][j])$   $\triangleright T$  is an  $N \times N$  anti-circulant matrix.
    for  $t_1 = 0$  to  $N - 1$  do
      for  $t_2 = 0$  to  $N - 1$  do
         $\mathbf{B}[i \cdot N + t_1][j \cdot N + t_2] \leftarrow \mathbf{T}[t_1][t_2]$ 
      end for
    end for
  end for
end for
return  $\mathbf{B}$ 

```

Vector sampling in $\mathbb{Z}_q^t$. Algorithm 7 is a vector sampling algorithm for the vector space $\mathbb{Z}_q^t$. We implement this algorithm using a 16-bit rejection sampling method. The algorithm takes as input a byte array of length $3t$ and outputs a vector $\mathbf{a} = (a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$. When $q = 8191$, $\text{threshold} = 8191 \times 8 = 65528$, the single rejection probability is $1 - 65528/2^{16} = 0.0001220703125$. When the input consists of $3t$ bytes, the maximum number of trials is $3t/2$, and

the probability of sampling failure is

$$1 - \sum_{i=t}^{3t/2} \binom{3t/2}{i} \left(\frac{8191}{8192}\right)^i \left(\frac{1}{8192}\right)^{3t/2-i}.$$

In the LoongKEM, the value of t is large, so the probability of sampling failure is much smaller than 2^{-256} .

Algorithm 7 `SampleVec(bytes, t)`: Vector sampling in $\mathbb{Z}_q^t$.

Input: `bytes` $\in \mathbb{B}^{3t}$ and t .

Output: $(a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$.

```

1:  $j \leftarrow 0$ 
2: for  $(i = 0; i < 3t \text{ and } j < t; i = i + 2)$  do
3:    $x \leftarrow \text{bytes}[i] + 2^8 \cdot \text{bytes}[i + 1]$ 
4:   if  $(x < \text{threshold})$  then
5:      $a_j \leftarrow x \bmod q$ 
6:      $j \leftarrow j + 1$ 
7:   end if
8: end for
9: return  $(a_0, a_1, \dots, a_{t-1})$ 

```

Sampling matrix without algebraic structure. Let

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0,k-1} \\ a_{10} & a_{11} & \cdots & a_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k-1,0} & a_{k-1,1} & \cdots & a_{k-1,k-1} \end{pmatrix} \in \mathbb{Z}_q^{k \times k}$$

be a matrix without algebraic structure, and we use a one-dimensional vector

$$\mathbf{a} = (a_{00}, \dots, a_{0,k-1}, a_{1,0}, \dots, a_{1,k-1}, \dots, a_{k-1,k-1}) \in \mathbb{Z}_q^{k^2}$$

to represent the matrix $\mathbf{A}$. The sampling algorithm for a matrix without algebraic structure is given in Algorithm 8. The algorithm takes as input a byte array `bytes` and a positive integer k , where the length of the array is $3k^2$, and outputs the matrix elements $\mathbf{a}$.

Algorithm 8 `SampleMat(bytes, k)`: Sampling matrix.

Input: k and `bytes` $\in \mathbb{B}^{3k^2}$.

Output: $\mathbf{a} = (a_{00}, \dots, a_{0,k-1}, a_{1,0}, \dots, a_{1,k-1}, \dots, a_{k-1,k-1}) \in \mathbb{Z}_q^{k^2}$.

```

1:  $\mathbf{a} \leftarrow \text{SampleVec}(\text{bytes}, k^2)$ 
2: return  $\mathbf{a}$ 

```

Polynomial matrix sampling. Let

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,v-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{u-1,0} & \mathbf{b}_{u-1,1} & \cdots & \mathbf{b}_{u-1,v-1} \end{pmatrix},$$

be a polynomial matrix, where $\mathbf{b}_{ij} \in R_q(0 \leq i < u, 0 \leq j \leq v-1)$. We arrange the coefficients of the polynomial into a one-dimensional vector:

$$\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}.$$

Then the polynomial matrix sampling algorithm is as follows:

Algorithm 9 SamplePolyMat(**bytes**, u, v): Polynomial matrix sampling.

Input: u, v and **bytes** $\in \mathbb{B}^{3uvN}$.

Output: $\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}$.

1: $\mathbf{b} \leftarrow \text{SampleVec}(\text{bytes}, uvN)$

2: **return** $\mathbf{b}$

Noise vector sampling. The coefficients of the noise polynomials in the LoongKEM are sampled according to the centered binomial distribution. Let η be a positive integer. Algorithm 10 is the noise vector sampling algorithm. The algorithm takes as input an array $\mathbf{b}$ of length $2t\eta/8$ bytes, and outputs a noise vector $\mathbf{e} \in \mathbb{Z}_q^t$.

Algorithm 10 SampleNoiseVec($\mathbf{b}, t$): Noise vector sampling.

Input: t and $\mathbf{b} \in \mathbb{B}^{2t\eta/8}$.

Output: $\mathbf{e} \in \mathbb{Z}_q^t$.

1: $(b_0, b_1, \dots, b_{2t\eta-1}) \leftarrow \text{BytesToBits}(\mathbf{b})$

2: **for** $(i = 0; i < t; i++)$ **do**

3: $a \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+j}$

4: $b \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+\eta+j}$

5: $e_i \leftarrow a - b$

6: **end for**

7: **return** $\mathbf{e} = (e_0, e_1, \dots, e_{t-1})$.

Noise matrix sampling. Let

$$\mathbf{E} = \begin{pmatrix} e_{00} & e_{01} & \cdots & e_{0,v-1} \\ e_{10} & e_{11} & \cdots & e_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ e_{u-1,0} & e_{u-1,1} & \cdots & e_{u-1,v-1} \end{pmatrix} \in \mathbb{Z}_q^{u \times v}$$

be a noise matrix. The entries of the matrix follow the centered binomial distribution. We can use a one-dimensional vector

$$\mathbf{e} = (e_{00}, \dots, e_{0,v-1}, e_{1,0}, \dots, e_{1,v-1}, \dots, e_{u-1,v-1}) \in \mathbb{Z}_q^{uv}$$

to represent the matrix $\mathbf{E}$. Then the sampling algorithm for the noise matrix is given in Algorithm 11. The algorithm takes as input a byte array **bytes** and positive integers u, v , where the length of the array is $2uv\eta/8$. It outputs the vector $\mathbf{e}$ composed of the entries of the noise matrix.

Algorithm 11 SampleNoiseMat(**bytes**, u, v): Noise matrix sampling.

Input: u, v and **bytes** $\in \mathbb{B}^{2uv\eta/8}$.

Output: $\mathbf{e} = (e_{00}, \dots, e_{0,v-1}, e_{1,0}, \dots, e_{1,v-1}, \dots, e_{u-1,v-1}) \in \mathbb{Z}_q^{uv}$.

1: $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\text{bytes}, uv)$

2: **return** $\mathbf{e}$

Noise polynomial vector sampling. Let $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_t) \in R^t$ be a noise polynomial vector, whose coefficients follow the centered binomial distribution. The sampling algorithm for the noise polynomial vector is given in Algorithm 12. The algorithm takes a byte array **bytes** as input and outputs the vector $\mathbf{e}$ composed of the coefficients of the noise polynomial vector.

Algorithm 12 SampleNoisePolyVec(**bytes**, t): Noise polynomial vector sampling.

Input: u, v and **bytes** $\in \mathbb{B}^{2tN\eta/8}$.

Output: $\mathbf{e} = (e_{00}, \dots, e_{0,N-1}, e_{1,0}, \dots, e_{1,N-1}, \dots, e_{t-1,N-1}) \in \mathbb{Z}_q^{tN}$.

1: $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\text{bytes}, tN)$

2: **return** $\mathbf{e}$

Encoding and decoding of polynomial vectors. Let the polynomial vector $\mathbf{b} = (\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \in R^k$, where the coefficients of the polynomials lie in the interval $[0, 2^d - 1]$. We regard the coefficients of the polynomial vector as a vector in the vector space $\mathbb{Z}^{kN}$. Then the encoding algorithm for $\mathbf{b}$ is given in Algorithm 13.

Algorithm 13 EncodePolyVec($\mathbf{b}, k, d$): Polynomial Vector Encoding.

Input: k, d and $\mathbf{b} \in R_q^k$.

Output: **bytes** $\in \mathbb{B}^{kNd/8}$.

1: **bytes** $\leftarrow \text{ByteEncode}(\mathbf{b}, kN, d)$

2: **return** **bytes**

The corresponding decoding algorithm is given in Algorithm 14.

Algorithm 14 DecodePolyVec(**bytes**, k, d): Polynomial Vector Decoding.

Input: k, d and **bytes** $\in \mathbb{B}^{kNd/8}$.

Output: $\mathbf{b} \in R^k$.

- 1: $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \leftarrow \text{ByteDecode}(\text{bytes}, kN, d)$
 - 2: **return** $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1})$
-

Encoding and decoding of general matrix. Let

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0,\ell-1} \\ a_{10} & a_{11} & \cdots & a_{1,\ell-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k-1,0} & a_{k-1,1} & \cdots & a_{k-1,\ell-1} \end{pmatrix} \in \mathbb{Z}_{2^d}^{k \times \ell}$$

be general matrix without algebraic structure, and $a_{ij} \in [0, 2^d - 1] (0 \leq i < k, 0 \leq j < \ell)$. We use a one-dimensional vector

$$\mathbf{a} = (a_{00}, \dots, a_{0,\ell-1}, a_{1,0}, \dots, a_{1,\ell-1}, \dots, a_{k-1,0}, \dots, a_{k-1,\ell-1}) \in \mathbb{Z}_{2^d}^{k\ell}$$

to represent the matrix $\mathbf{A}$, Then the encoding of an ordinary matrix without algebraic structure is given in Algorithm 15.

Algorithm 15 EncodeMat($\mathbf{a}, k, \ell, d$): General matrix encoding.

Input: $\mathbf{a} \in \mathbb{Z}_{2^d}^{k\ell}$, k, ℓ and d .

Output: **bytes** $\in \mathbb{B}^{k\ell d/8}$.

- 1: **bytes** $\leftarrow \text{ByteEncode}(\mathbf{a}, k\ell, d)$
 - 2: **return** **bytes**
-

The corresponding decoding algorithm is presented in Algorithm 16.

Algorithm 16 DecodeMat(**bytes**, k, ℓ, d): General matrix decoding.

Input: **bytes** $\in \mathbb{B}^{k\ell d/8}$, k, ℓ and d .

Output: $\mathbf{a} \in \mathbb{Z}_{2^d}^{k\ell}$.

- 1: $\mathbf{a} \leftarrow \text{ByteDecode}(\text{bytes}, k\ell, d)$
 - 2: **return** $\mathbf{a}$
-

Encoding and decoding of noise polynomial vectors Let $\mathbf{e} = (\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{k-1}) \in R^k$ denote a polynomial vector such that $\|\mathbf{e}_i\|_\infty \leq \eta \leq 6$ for all $0 \leq i < k$. Define $d_\eta = 4$. We flatten all coefficients of the polynomial vector into a one-dimensional array $(e_0, e_1, \dots, e_{kN-1}) \in \mathbb{Z}^{kN}$ ordered by subscript indices. The encoding algorithm for noise polynomial vectors is presented in Algorithm 17, and Algorithm 18 corresponds to the decoding procedure for noise polynomials.

Auxiliary cryptographic functions. The Loong (KEM) requires the use of cryptographically secure pseudorandom function RNG, hash functions H and G, and extendable-output function XOF.

Algorithm 17 EncodeNoisePolyVec($\mathbf{e}, k, d_\eta$)

Input: Positive integers d_η, k ; coefficients of the noise polynomial vector $(e_0, e_1, \dots, e_{kN-1})$

Output: Byte array **bytes**.

- 1: $\mathbf{b} \leftarrow \{0, 0, \dots, 0\}$ $\triangleright$ The length of $\mathbf{b}$ is kN .
 - 2: **for** $i = 0$ to $kN - 1$ **do**
 - 3: $\mathbf{b}[i] \leftarrow e_i + \eta$
 - 4: **end for**
 - 5: $\mathbf{bytes} \leftarrow \text{ByteEncode}(\mathbf{b}, kN, d_\eta)$
 - 6: **return** **bytes**
-

Algorithm 18 DecodeNoisePolyVec(**bytes**, k, η)

Input: Byte array **bytes**; positive integers k, d_η .

Output: Array $\mathbf{e}$ of length kN satisfying $\|\mathbf{e}\|_\infty \leq \eta$.

- 1: $\mathbf{a} \leftarrow \text{ByteDecode}(\mathbf{bytes}, kN, d_\eta)$
 - 2: **for** $i = 0$ to $kN - 1$ **do**
 - 3: $e_i \leftarrow \mathbf{a}[i] - \eta$
 - 4: **end for**
 - 5: **return** $\mathbf{e} = (e_0, e_1, \dots, e_{kN-1})$
-

2.2 Loong public key encryption

This subsection presents the CPA-secure Loong public-key encryption algorithm.

2.2.1 Keypair generation

Algorithm 19 is the key pair generation algorithm for the Loong public-key encryption scheme.

Algorithm 19 Loong_PKE.KeyGen(): Key generation algorithm

Input: NULL.

Output: pk and sk.

```

1:  $seed \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$ 
2:  $(\rho, \sigma, \gamma) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \leftarrow \text{XOF}(seed)$ 
3:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3(k_1^2N + 2k_1k_2N + k_2^2N^2).$ 
4:  $\mathbf{A}_1 \leftarrow \text{SamplePolyMat}(\mathbf{x}, k_1, k_1)$   $\triangleright \mathbf{A}_1 \in R_q^{k_1 \times k_1}.$ 
5:  $\mathbf{A}_2 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3k_1^2N, k_1, k_2)$   $\triangleright \mathbf{A}_2 \in R_q^{k_1 \times k_2}.$ 
6:  $\mathbf{A}_3 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3(k_1^2N + k_1k_2N), k_2, k_1)$   $\triangleright \mathbf{A}_3 \in R_q^{k_2 \times k_1}.$ 
7:  $\mathbf{A}_4 \leftarrow \text{SampleMat}(\mathbf{x} + 3(k_1^2N + 2k_1k_2N), k_2N, k_2N)$   $\triangleright \mathbf{A}_4 \in \mathbb{Z}_q^{k_2N \times k_2N}.$ 
8:  $\mathbf{y} \leftarrow \text{XOF}(\sigma)$   $\triangleright |\mathbf{y}| = 2(k_1N + k_2N)\eta/8.$ 
9:  $\mathbf{s}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k_1)$   $\triangleright \mathbf{s}_1 \in R_q^{k_1}.$ 
10:  $\mathbf{s}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y} + 2k_1N\eta/8, k_2)$   $\triangleright \mathbf{s}_2 \in R_q^{k_2}.$ 
11:  $\mathbf{S}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{s}_2, k_2, 1)$   $\triangleright \mathbf{S}_2 \in \mathbb{Z}_q^{k_2N \times N}.$ 
12:  $\mathbf{z} \leftarrow \text{XOF}(\gamma)$   $\triangleright |\mathbf{z}| = 2(k_1N + k_2N^2)\eta/8.$ 
13:  $\mathbf{e}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k_1)$   $\triangleright \mathbf{e}_1 \in R_q^{k_1}.$ 
14:  $\mathbf{E}_2 \leftarrow \text{SampleNoiseMat}(\mathbf{z} + 2k_1N\eta/8, k_2N, N)$   $\triangleright \mathbf{E}_2 \in \mathbb{Z}_q^{k_2N \times N}.$ 
15:  $\mathbf{b}_1 \leftarrow \mathbf{A}_1\mathbf{s}_1 + \mathbf{A}_2\mathbf{s}_2 + \mathbf{e}_1$   $\triangleright \mathbf{b}_1 \in R_q^{k_1}.$ 
16:  $\mathbf{B}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{A}_3\mathbf{s}_1, k_2, 1) + \mathbf{A}_4\mathbf{S}_2 + \mathbf{E}_2$   $\triangleright \mathbf{B}_2 \in \mathbb{Z}_q^{k_2N \times N}.$ 
17:  $\mathbf{b} \leftarrow \text{Compress}(\mathbf{b}_1, 2^{d_q-d_b})$ 
18:  $\mathbf{B} \leftarrow \text{Compress}(\mathbf{B}_2, 2^{d_q-d_b})$ 
19:  $\text{pk} \leftarrow \rho || \text{EncodePolyVec}(\mathbf{b}, k_1, d_b) || \text{EncodeMat}(\mathbf{B}, k_2N, N, d_b)$ 
20:  $\text{sk} \leftarrow \text{EncodeNoisePolyVec}(\mathbf{s}_1, k_1, d_\eta) || \text{EncodeNoisePolyVec}(\mathbf{s}_2, k_2, d_\eta)$ 
21: return (pk, sk)
```

2.2.2 Encryption

Algorithm 20 is the Loong encryption algorithm.

Algorithm 20 Loong_PKE.Encrypt(pk, M, γ): Loong encryption

Input: pk, $\mathbf{M} \in \{0, 1\}^{N \times N}$ and $\gamma = (\gamma_1, \gamma_2, \gamma_3) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16}$.

Output: Ciphertext $\mathbf{c}$.

```

1:  $\rho \leftarrow \text{pk}[0 : \lambda/8 + 16 - 1]$ 
2:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3(k_1^2 N + 2k_1 k_2 N + k_2^2 N^2)$ .
3:  $\mathbf{A}_1 \leftarrow \text{SamplePolyMat}(\mathbf{x}, k_1, k_1)$   $\triangleright \mathbf{A}_1 \in R_q^{k_1 \times k_1}$ .
4:  $\mathbf{A}_2 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3k_1^2 N, k_1, k_2)$   $\triangleright \mathbf{A}_2 \in R_q^{k_1 \times k_2}$ .
5:  $\mathbf{A}_3 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3(k_1^2 N + k_1 k_2 N), k_2, k_1)$   $\triangleright \mathbf{A}_3 \in R_q^{k_2 \times k_1}$ .
6:  $\mathbf{A}_4 \leftarrow \text{SampleMat}(\mathbf{x} + 3(k_1^2 N + 2k_1 k_2 N), k_2 N, k_2 N)$   $\triangleright \mathbf{A}_4 \in \mathbb{Z}_q^{k_2 N \times k_2 N}$ .
7:  $\tilde{\mathbf{b}}_1 \leftarrow \text{DecodePolyVec}(\text{pk}[\lambda/8 + 16 : \lambda/8 + 16 + k N d_b / 8 - 1], k_1, d_b)$ 
8:  $\mathbf{b}_1 \leftarrow \text{Decompress}(\tilde{\mathbf{b}}_1, 2^{d_q - d_b})$   $\triangleright \mathbf{b}_1 \in R_q^{k_1}$ .
9:  $\tilde{\mathbf{B}}_2 \leftarrow \text{DecodeMat}(\text{pk}[\lambda/8 + 16 + k N d_b / 8 - 1 : \#pk], k_2 N, k_2 N, d_b)$ 
10:  $\mathbf{B}_2 \leftarrow \text{Decompress}(\tilde{\mathbf{B}}_2, 2^{d_q - d_b})$   $\triangleright \mathbf{B}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ .
11:  $\mathbf{w} \leftarrow \text{XOF}(\gamma_1)$   $\triangleright |\mathbf{w}| = 2(k_1 N + k_2 N)\eta/8$ .
12:  $\mathbf{r}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w}, k_1)$   $\triangleright \mathbf{r}_1 \in R_q^{k_1}$ .
13:  $\mathbf{r}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w} + 2k_1 N \eta / 8, k_2)$   $\triangleright \mathbf{r}_2 \in R_q^{k_2}$ .
14:  $\mathbf{R}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{r}_2^T, 1, k_2)$   $\triangleright \mathbf{R}_2 \in \mathbb{Z}_q^{N \times k_2 N}$ .
15:  $\mathbf{y} \leftarrow \text{XOF}(\gamma_2)$   $\triangleright |\mathbf{y}| = 2(k_1 N + k_2 N^2)\eta/8$ .
16:  $\mathbf{e}_3 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k_1)$   $\triangleright \mathbf{e}_3 \in R_q^{k_1}$ .
17:  $\mathbf{E}_4 \leftarrow \text{SampleNoiseMat}(\mathbf{y} + 2k_1 N \eta / 8, N, k_2 N)$   $\triangleright \mathbf{E}_4 \in \mathbb{Z}_q^{N \times k_2 N}$ .
18:  $\mathbf{u}' \leftarrow \mathbf{r}_1^T \mathbf{A}_1 + \mathbf{r}_2^T \mathbf{A}_3 + \mathbf{e}_3$   $\triangleright \mathbf{u}' \in R_q^{k_1}$ .
19:  $\mathbf{U}' \leftarrow \text{PolysToBlockMat}(\mathbf{r}_1^T \mathbf{A}_2, 1, k_2) + \mathbf{R}_2 \mathbf{A}_4 + \mathbf{E}_4$   $\triangleright \mathbf{U}' \in \mathbb{Z}_q^{N \times k_2 N}$ .
20:  $\mathbf{z} \leftarrow \text{XOF}(\gamma_3)$   $\triangleright |\mathbf{z}| = 2N^2 \eta / 8$ .
21:  $\mathbf{E}_5 \leftarrow \text{SampleNoiseMat}(\mathbf{z}, N, N)$   $\triangleright \mathbf{E}_5 \in \mathbb{Z}_q^{N \times N}$ .
22:  $\mathbf{V}' \leftarrow \text{PolyToMat}(\mathbf{r}_1^T \mathbf{b}_1) + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \frac{q+1}{2} \cdot \mathbf{M}$   $\triangleright \mathbf{V}' \in \mathbb{Z}_q^{N \times N}$ .
23:  $\mathbf{u} \leftarrow \text{Compress}(\mathbf{u}', 2^{d_q - d_u})$ 
24:  $\mathbf{U} \leftarrow \text{Compress}(\mathbf{U}', 2^{d_q - d_u})$ 
25:  $\mathbf{V} \leftarrow \text{Compress}(\mathbf{V}', 2^{d_q - d_v})$ 
26:  $\mathbf{c} \leftarrow \text{EncodePolyVec}(\mathbf{u}, k_1, d_u) || \text{EncodeMat}(\mathbf{U}, N, k_2 N, d_u) || \text{EncodeMat}(\mathbf{V}, N, N, d_v)$ 
27: return  $\mathbf{c}$ .
```

2.2.3 Decryption

Algorithm 21 is the Loong decryption algorithm. The decryption process is similar to NTRU.

Algorithm 21 Loong_PKE.Decrypt(sk, c): Loong decryption

Input: sk and c.

Output: Plaintext M.

```

1:  $\mathbf{u} \leftarrow \text{DecodePolyVec}(\mathbf{c}, k_1, d_u)$   $\triangleright \mathbf{u} \in R_q^{k_1}$ .
2:  $\mathbf{U} \leftarrow \text{DecodeMat}(\mathbf{c} + k_1 N d_u / 8, N, k_2 N, d_u)$   $\triangleright \mathbf{U} \in \mathbb{Z}_q^{N \times k_2 N}$ .
3:  $\mathbf{V} \leftarrow \text{DecodeMat}(\mathbf{c} + (k_1 N + k_2 N^2) d_u / 8, N, N, d_v)$   $\triangleright \mathbf{V} \in \mathbb{Z}_q^{N \times N}$ .
4:  $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \text{DecodeNoisePolyVec}(\text{sk}, k_1 + k_2, d_\eta)$   $\triangleright \mathbf{s}_1 \in R_q^{k_1}, \mathbf{s}_2 \in R_q^{k_2}$ .
5:  $\mathbf{S}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{s}_2, k_2, 1)$   $\triangleright \mathbf{S}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ .
6:  $\mathbf{u}' \leftarrow \text{Decompress}(\mathbf{u}, 2^{d_q - d_u})$ .
7:  $\mathbf{U}' \leftarrow \text{Decompress}(\mathbf{U}, 2^{d_q - d_u})$ .
8:  $\mathbf{V}' \leftarrow \text{Decompress}(\mathbf{V}, 2^{d_q - d_v})$ .
9:  $\mathbf{W} \leftarrow \text{PolyToMat}(\mathbf{u}'^T \mathbf{s}_1) + \mathbf{U}' \mathbf{S}_2$   $\triangleright \mathbf{W} \in \mathbb{Z}_q^{N \times N}$ .
10:  $\mathbf{M} \leftarrow (2 \cdot (\mathbf{V}' - \mathbf{W}) \bmod \pm q) \bmod 2$ 
11: return M

```

Correctness Since $\mathbf{u}', \mathbf{U}', \mathbf{V}', \mathbf{b}_1, \mathbf{B}_2$ are values obtained after applying the compress and decompress functions, from the properties of compress and decompress functions, we have

$$\begin{aligned}
\mathbf{V}' &= \text{PolyToMat}(\mathbf{r}_1^T \mathbf{b}_1) + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \frac{q+1}{2} \cdot \mathbf{M} + \mathbf{E}_6, \\
\mathbf{u}' &= \mathbf{r}_1^T \mathbf{A}_1 + \mathbf{r}_2^T \mathbf{A}_3 + \mathbf{e}_3 + \mathbf{e}_7, \\
\mathbf{U}' &= \text{PolysToBlockMat}(\mathbf{r}_1^T \mathbf{A}_2, 1, k_2) + \mathbf{R}_2 \mathbf{A}_4 + \mathbf{E}_4 + \mathbf{E}_8, \\
\mathbf{b}_1 &= \mathbf{A}_1 \mathbf{s}_1 + \mathbf{A}_2 \mathbf{s}_2 + \mathbf{e}_1 + \mathbf{e}_9, \\
\mathbf{B}_2 &= \text{PolysToBlockMat}(\mathbf{A}_3 \mathbf{s}_1, k_2, 1) + \mathbf{A}_4 \mathbf{s}_2 + \mathbf{E}_2 + \mathbf{E}_{10},
\end{aligned}$$

where the infinity norms of error terms satisfy $\|\mathbf{E}_6\|_\infty \leq 2^{d_q - d_v - 1}$, $\|\mathbf{e}_7\|_\infty \leq 2^{d_q - d_u - 1}$, $\|\mathbf{E}_8\|_\infty \leq 2^{d_q - d_u - 1}$, $\|\mathbf{e}_9\|_\infty \leq 2^{d_q - d_b - 1}$, $\|\mathbf{E}_{10}\|_\infty \leq 2^{d_q - d_b - 1}$. Let $\mathbf{S}_1, \mathbf{R}_1, \mathbf{R}_2, \mathbf{E}_3, \mathbf{E}_7, \mathbf{B}_1$ be the matrices corresponding to polynomial vectors $\mathbf{s}_1, \mathbf{r}_1^T, \mathbf{r}_2^T, \mathbf{e}_3, \mathbf{e}_7, \mathbf{b}_1$, respectively. Thus we have

$$\begin{aligned}
& 2 \cdot (\mathbf{V}' - \mathbf{W}) \bmod \pm q \\
&= 2 \cdot (\mathbf{R}_1 \mathbf{B}_1 + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \Delta \mathbf{M} + \mathbf{E}_6 - \mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 - \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1) \\
&\quad - 2 \cdot (\mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 - \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) \bmod \pm q \\
&= 2 \cdot (\mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 + \mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 + \mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 + \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2 + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10})) \\
&\quad + 2 \cdot (\mathbf{E}_5 + \Delta \mathbf{M} + \mathbf{E}_6 - \mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 - \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - \mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 - \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2) \\
&\quad - 2 \cdot ((\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) \bmod \pm q \\
&= 2 \cdot (\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) + \mathbf{M} \bmod \pm q
\end{aligned}$$

If the following infinity norm condition holds:

$$\left\| 2 \cdot \left(\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2 \right) + \mathbf{M} \right\|_\infty < \frac{q}{2},$$

then the signed modulo- q operation reduces to the original expression, i.e.,

$$\begin{aligned}
& 2 \cdot (\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) + \mathbf{M} \bmod \pm q \\
&= 2 \cdot (\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) + \mathbf{M}.
\end{aligned}$$

Therefore, we can successfully decrypt the ciphertext.

Probability of Decryption Failure If

$$\|\mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2\|_\infty \geq \frac{q}{4},$$

then decryption failure. We now compute the probability of decryption failure. Define

$$\mathbf{E} = \mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2.$$

The matrix $\mathbf{E}$ can be rewritten as

$$\mathbf{E} = (\mathbf{R}_1 \quad \mathbf{R}_2) \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix} + (\mathbf{R}_1 \quad \mathbf{R}_2) \begin{pmatrix} \mathbf{E}_9 \\ \mathbf{E}_{10} \end{pmatrix} - (\mathbf{E}_3 \quad \mathbf{E}_4) \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} - (\mathbf{E}_7 \quad \mathbf{E}_8) \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \mathbf{E}_5 + \mathbf{E}_6.$$

Clearly, every entry of $\mathbf{E}$ takes the form

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle + e_5 + e_6,$$

where

- $\hat{\mathbf{r}}, \hat{\mathbf{s}}, \hat{\mathbf{e}}_1, \hat{\mathbf{e}}_3$ are $n = (k_1 + k_2)N$ -dimensional vectors whose entries follow the centered binomial distribution ψ_η ;
- $\hat{\mathbf{e}}_2, \hat{\mathbf{e}}_4$ are $n = (k_1 + k_2)N$ -dimensional vectors. We choose appropriate values of d_u, d_b such that each entry of $\hat{\mathbf{e}}_2, \hat{\mathbf{e}}_4$ lies in the set $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$;
- e_5 follows the centered binomial distribution ψ_η ;
- $|e_6|_\infty \leq 2^{d_q - d_v - 1}$.

Decryption fails if some entry of $\mathbf{E}$ satisfies

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle + e_5 + e_6 \geq \frac{q}{4},$$

or equivalently if

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle \geq \frac{q}{4} - 2^{d_q - d_v - 1} - \eta.$$

Assume the vectors $\hat{\mathbf{r}}, \hat{\mathbf{s}}, \hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3, \hat{\mathbf{e}}_4$ follow a zero-mean Gaussian distribution with standard deviation σ . Then the quantity

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle$$

is approximately Gaussian with zero mean and standard deviation $\sqrt{4n}\sigma$. The probability that

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle \geq \frac{q}{4} - 2^{d_q - d_v - 1} - \eta$$

holds is approximately

$$1 - \text{erf} \left(\frac{\frac{q}{4} - 2^{d_q - d_v - 1} - \eta}{\sqrt{2} \cdot \sqrt{4n} \sigma} \right),$$

where the error function is defined as $\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$ and $\sigma^2 = \eta/2$. Therefore, the overall probability of decryption failure is approximately

$$n \cdot \left(1 - \text{erf} \left(\frac{\frac{q}{4} - 2^{d_q - d_v - 1} - \eta}{\sqrt{2} \cdot \sqrt{4n} \sigma} \right) \right).$$

2.3 LoongKEM

We use the Fujisaki-Okamoto transform [12] to convert the Loong encryption algorithm into a key encapsulation mechanism. Algorithm 22 is the key generation algorithm, where $\text{pk}_{\text{size}} = (\lambda + k_1 N d_b + k_2 N^2 d_b)/8 + 16$, $\text{sk}_{\text{size}} = (k_1 + k_2)N/2 + \text{pk}_{\text{size}} + |\mathbb{H}(pk)| + \lambda/8 + 16$. Algorithm 23 performs key encapsulation, where ctx_{size} denotes the byte length of the ciphertext: $\text{ctx}_{\text{size}} = (k_1 N d_u + k_2 N^2 d_u + N^2 d_v)/8$. Algorithm 24 performs key decapsulation.

Algorithm 22 Loong_KEM_KeyGen()

Input: NULL

Output: $\text{pk} \in \mathbb{B}^{\text{pk}_{\text{size}}}$ and $\text{sk} \in \mathbb{B}^{\text{sk}_{\text{size}}}$

- 1: $\text{salt} \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$
 - 2: $(\text{pk}, \text{sk}') \leftarrow \text{Loong_PKE_KeyGen}()$ ▷ Execute the algorithm 19.
 - 3: $\text{sk} \leftarrow (\text{sk}' \parallel \text{pk} \parallel \mathbb{H}(\text{pk}) \parallel \text{salt})$
 - 4: **return** (pk, sk)
-

Algorithm 23 Loong_KEM_Encaps(pk)

Input: $\text{pk} \in \mathbb{B}^{\text{pk}_{\text{size}}}$.

Output: Share key $K \in \mathbb{B}^{\lambda/8}$ and ciphertext $\mathbf{c} \in \mathbb{B}^{\text{ctx}_{\text{size}}}$.

- 1: $\mathbf{M} \in \{0, 1\}^{N \times N} \leftarrow \text{RNG}()$
 - 2: $(K, \gamma) \leftarrow \mathbf{G}(\mathbf{M} \parallel \mathbb{H}(\text{pk}))$
 - 3: $\mathbf{c} \leftarrow \text{Loong_PKE_Encrypt}(\text{pk}, \mathbf{M}, \gamma)$
 - 4: **return** $(K, \mathbf{c})$
-

Algorithm 24 Loong_KEM.Decaps(sk, c)

Input: $\text{sk} \in \mathbb{B}^{\text{sk}_{\text{size}}}$ and $\text{c} \in \mathbb{B}^{\text{ctx}_{\text{size}}}$ **Output:** Share key $K \in \mathbb{B}^{\lambda/8}$

- 1: $(\text{sk}', \text{pk}, \text{h}, \text{salt}) \leftarrow \text{sk}$
 - 2: $\text{M}' \leftarrow \text{Loong_PKE.Decrypt}(\text{sk}', \text{c})$
 - 3: $(K', \gamma') \leftarrow \text{G}(\text{M}' \parallel \text{h})$
 - 4: $\bar{K} \leftarrow \text{H}(\text{salt} \parallel \text{c})$
 - 5: $\text{c}' \leftarrow \text{Loong_PKE.Encrypt}(\text{pk}, \text{M}', \gamma')$
 - 6: **if** $\text{c} \neq \text{c}'$ **then**
 - 7: $K' \leftarrow \bar{K}$
 - 8: **end if**
 - 9: **return** K'
-

2.4 Parameters

After comprehensively considering security, performance, and decryption failure probability, we select the parameters for the LoongKEM as shown in Table 2.

	λ	q	N	(k_1, k_2)	η	(d_b, d_u, d_v)	δ
Loong128	128	8191	12	(48, 4)	5	(10, 10, 4)	2^{-141}
Loong256	256	8191	16	(64, 6)	4	(10, 10, 10)	2^{-161}
Loong384	384	8191	20	(72, 8)	3	(11, 11, 6)	2^{-190}
Loong512	512	8191	24	(80, 10)	2	(11, 11, 4)	2^{-260}

Table 2: Parameters of the LoongKEM.

The parameter λ denotes the classical bit security level of the key encapsulation algorithm. The corresponding encapsulated key lengths and security levels of the Loong128, Loong256, Loong384, and Loong512 key encapsulation algorithms are 128 bits, 256 bits, 384 bits, and 512 bits, respectively. The parameter $q = 8191 = 2^{13} - 1$ is a Mersenne prime, which allows fast computation of modulo- q operations. The parameter N is the degree of the modulus polynomial; the elements of polynomial matrices and polynomial vectors are polynomials of degree less than N . k_1 and k_2 are parameters related to the dimensions of polynomial matrices and polynomial vectors. η is the parameter of the central binomial distribution. d_b is the public-key compression parameter, while d_u and d_v are the ciphertext compression parameters. δ denotes the probability of key decapsulation failure.

Explanation of Parameter Selection The security of LoongKEM is based on the semi-structured Learning With Errors (SLWE) problem. The reasons for choosing the values of N are as follows: On the one hand, the value of the parameter N should not be too large; otherwise, LoongKEM would possess strong algebraic structures similar to those of Kyber. On the other hand, N cannot be too small. We require an $N \times N$ matrix to carry at least λ bits of information.

In this way, only a single matrix is needed to store key material, which improves the performance and efficiency of the key encapsulation mechanism, and reduces storage overhead and transmission bandwidth consumption.

We select $q = 8191 = 2^{13} - 1$ for the following reasons. First, 8191 is a Mersenne prime, enabling fast modular arithmetic modulo q . Second, q must be chosen such that the probability of decryption failure remains negligible. Based on the derived formula for decryption failure probability, we set $q = 8191$.

The choices of parameters k_1 and k_2 are governed by the considerations below. First, the dimension of the public-key matrix is $(k_1 + k_2)N$. After fixing q and N , we must select suitable k_1, k_2 to guarantee that LoongKEM meets the target security level. We compute the minimal admissible $(k_1 + k_2)N$ satisfying security requirements via the security evaluation formulas in Subsection 5, with sufficient security margin for conservative estimation. Moreover, k_2 controls the degree of unstructuredness in the public-key matrix and ciphertext. We enforce the constraint $(2\eta + 1)^{k_2} > 2^\lambda$, which renders brute-force elimination of the unstructured components in public keys and ciphertexts computationally infeasible for adversaries.

The value of η is determined jointly by the security evaluation formulas in section 5 and the decryption failure probability formula, ensuring LoongKEM achieves the target security while maintaining a low decryption failure probability.

The triplet (d_b, d_u, d_v) is primarily determined by the decryption failure probability formula.

3 Design principle

The design principle of the LoongKEM key encapsulation mechanism shares similarities with Kyber and FrodoKEM. The main differences between LoongKEM and Kyber, FrodoKEM are listed as follows:

- The decryption logic differs. The final step of the decryption algorithms for Kyber and FrodoKEM generally compares the ciphertext against $q/4$: if the value is greater than $q/4$, the corresponding plaintext bit is set to 1; otherwise, the plaintext bit is 0. In the Loong encryption algorithm, plaintext can be recovered merely via modular arithmetic operations. This procedure resembles the decryption process of the NTRU encryption scheme and accelerates decryption speed.
- The **Compress** and **Decompress** methods for public keys and ciphertexts are distinct. LoongKEM realizes compression and decompression of elements in $\mathbb{Z}_q$ solely through shift and addition operations, without division or modular operations.
- The underlying hard problems differ. Kyber relies on the hardness of the MLWE problem, FrodoKEM is based on the hardness of the LWE problem, while LoongKEM is built upon the hardness of the semi-structured LWE problem (SLWE problem).

3.1 Semi-structured LWE problem

The security of LoongKEM is based on the semi-structured LWE problem (SLWE for short). We now give the definition of the SLWE problem. Let N, q, u, v be positive integer. Let $\phi(X) \in \mathbb{Z}[X]$ be a polynomial of degree N . Let $\widehat{R} = \mathbb{Z}[X]/\phi(X)$ be the polynomial ring. For any $\mathbf{f}(X) = f_0 + f_1X + \dots + f_{N-1}X^{N-1} \in \widehat{R}$, define the corresponding matrix of $\mathbf{f}(X)$ as $\mathbf{M}_{\mathbf{f}}$, where the i -th $i(0 \leq i < N)$ row of $\mathbf{M}_{\mathbf{f}}$ is the coefficient vector of the polynomial $X^i \mathbf{f}(X) \bmod \phi(X)$. Let the polynomial matrix

$$\mathbf{F} = \begin{pmatrix} \mathbf{f}_{00} & \mathbf{f}_{01} & \cdots & \mathbf{f}_{0,v-1} \\ \mathbf{f}_{10} & \mathbf{f}_{11} & \cdots & \mathbf{f}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{f}_{u-1,0} & \mathbf{f}_{u-1,1} & \cdots & \mathbf{f}_{u-1,v-1} \end{pmatrix} \in \widehat{R}^{u \times v},$$

where $\mathbf{f}_{ij} \in \widehat{R}(0 \leq i \leq u-1, 0 \leq j \leq v-1)$. We define the integer matrix corresponding to the polynomial matrix $\mathbf{F}$ as

$$\mathbf{M}_{\mathbf{F}} = \begin{pmatrix} \mathbf{M}_{\mathbf{f}_{00}} & \mathbf{M}_{\mathbf{f}_{01}} & \cdots & \mathbf{M}_{\mathbf{f}_{0,v-1}} \\ \mathbf{M}_{\mathbf{f}_{10}} & \mathbf{M}_{\mathbf{f}_{11}} & \cdots & \mathbf{M}_{\mathbf{f}_{1,v-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{M}_{\mathbf{f}_{u-1,0}} & \mathbf{M}_{\mathbf{f}_{u-1,1}} & \cdots & \mathbf{M}_{\mathbf{f}_{u-1,v-1}} \end{pmatrix} \in \mathbb{Z}^{uN \times vN},$$

where $\mathbf{M}_{\mathbf{f}_{ij}}$ is the matrix corresponding to the polynomial $\mathbf{f}_{ij}$.

Let k_1, k_2, k_3 be positive integers, and let the matrix $\mathbf{A} = \begin{pmatrix} \mathbf{M}_{\mathbf{A}_1} & \mathbf{M}_{\mathbf{A}_2} \\ \mathbf{M}_{\mathbf{A}_3} & \mathbf{A}_4 \end{pmatrix}$, where $\mathbf{M}_{\mathbf{A}_1}, \mathbf{M}_{\mathbf{A}_2}, \mathbf{M}_{\mathbf{A}_3}$ are the integer matrices corresponding to the polynomial matrices $\mathbf{A}_1 \in \widehat{R}^{k_1 \times k_1}, \mathbf{A}_2 \in \widehat{R}^{k_1 \times k_2}, \mathbf{A}_3 \in \widehat{R}^{k_3 \times k_1}$, respectively. $\mathbf{A}_4 \in \mathbb{Z}^{k_3N \times k_2N}$ is a generic integer matrix without algebraic structure. The entries of matrix $\mathbf{A}$ follow the uniform distribution.

Let χ be a probability distribution defined over $\mathbb{Z}$. Let $\mathbf{S} = \begin{pmatrix} \mathbf{M}_{\mathbf{S}_1} \\ \mathbf{M}_{\mathbf{S}_2} \end{pmatrix}$, $\mathbf{E} = \begin{pmatrix} \mathbf{M}_{\mathbf{E}_1} \\ \mathbf{E}_2 \end{pmatrix}$, where $\mathbf{M}_{\mathbf{S}_1}, \mathbf{M}_{\mathbf{S}_2}, \mathbf{M}_{\mathbf{E}_1}$ are the integer matrices corresponding to the polynomial matrices $\mathbf{S}_1 \in \widehat{R}^{k_1 \times 1}, \mathbf{S}_2 \in \widehat{R}^{k_2 \times 1}, \mathbf{E}_1 \in \widehat{R}^{k_1 \times 1}$, respectively. $\mathbf{E}_2$ is a matrix of size $k_3N \times N$ without algebraic structure. The entries of matrices $\mathbf{S}$ and $\mathbf{E}$ follow the distribution χ .

Definition 1 (SLWE problem). *Let $\mathbf{B} = \mathbf{AS} + \mathbf{E} \bmod q$, the SLWE problem is defined as follows: given matrices $\mathbf{A}$ and $\mathbf{B}$, find the matrix $\mathbf{S}$.*

Compared with cryptographic schemes based on the LWE problem, the SLWE problem incorporates algebraic structure partially, thereby reducing the lengths of public keys and ciphertexts. In contrast to the MLWE problem and RLWE problem, the algebraic structure introduced by the SLWE problem is relatively weaker. By striking a balance between security and efficiency, the SLWE problem is therefore adopted to construct our cryptographic algorithm.

3.2 Error distribution

In cryptosystems based on the LWE problem, the following noise distributions are commonly used:

- Discrete Gaussian distribution, e.g., FrodoKEM[13].
- Centered binomial distribution, e.g., Kyber[10] and Dilithium[11].
- NTRU-type distribution, where the coefficients of the polynomial belong to $\{-1, 0, 1\}$ with fixed numbers of -1, 0, and 1, e.g., the NTRU encryption algorithm[2].
- Uniform distribution, e.g., Saber[14].

Since the centered binomial distribution can be efficiently implemented in constant time and provides better resistance against side-channel attacks, we use it to sample noise polynomials and noise matrices.

4 Performance analysis

4.1 Performance

In our reference implementation, only the seed bits for generating the public-key matrix $\mathbf{A}$ are stored for public and secret key storage, with a length of $\lambda/8 + 16$ bytes. When bandwidth and storage space are not constrained, the entire entries of the public-key matrix $\mathbf{A}$ can be stored to accelerate key encapsulation and decapsulation.

The performance of the reference implementation and the AVX2 implementation of the LoongKEM is shown in Table 3.

Loong128		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 1832	gen : 2621	gen : 2602
pk : 1472	enc : 2647	enc : 2617
ct : 1512	dec : 2676	dec : 2636
Loong256		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 3888	gen : 6483	gen : 6426
pk : 3248	enc : 6552	enc : 6477
ct : 3520	dec : 6645	dec : 6531
Loong384		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 7365	gen : 20555	gen : 20531
pk : 6444	enc : 20656	enc : 20543
ct : 6680	dec : 20843	dec : 20795
Loong512		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 11864	gen : 33405	gen : 33331
pk : 10640	enc : 33756	enc : 35058
ct : 10848	dec : 34006	dec : 33736

Table 3: Performance of the LoongKEM.

Test environment for the reference implementation and AVX2 implementation is showed as follows: CPU model: Intel Core i7-1065G7, CPU frequency: 1.30 GHz; memory size of the performance test machine: 8 GB; operating system: Ubuntu 24.04.3 LTS, Linux kernel version: 6.14.0; compiler version: gcc 13.3.0; compilation flags: `-O3 -march=native -fomit-frame-pointer`. The number of cycles in the table is the average value obtained by executing the relevant functions 10000 times. Neither our reference implementation nor the AVX2 implementation uses heap memory.

4.2 Known answer test values

Using the auxiliary programs and data provided by the Commercial Cryptography Standard Research Institute, we have generated test vectors for all algorithm instances. These test vectors are placed in the `Test_Vector` directory of the submission package, with the specific test vector files as follows:

```

/ Test_Vector/KAT_KEM.Loong128.txt
/ Test_Vector/KAT_KEM.Loong256.txt
/ Test_Vector/KAT_KEM.Loong384.txt
/ Test_Vector/KAT_KEM.Loong512.txt

```

5 Security analysis

5.1 Security proof

Let XOF , H and G be random oracles, the decision SLWE problem be hard, and the public-key matrices $\mathbf{A}, \mathbf{B}$ remain uncompressed. We first prove that LoongPKE is IND-CPA security.

Theorem 1. *If the decision SLWE problem is hard, then the LoongPKE public-key encryption scheme is IND-CPA security.*

Proof. Suppose there exists a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that wins the IND-CPA game with non-negligible advantage ϵ . We construct a simulator $\mathcal{B}$ to solve the decision SLWE problem. Assume $\mathcal{B}$ receives a decision SLWE challenge sample $(\mathbf{A}, \mathbf{B})$, where $\mathbf{B}$ is either drawn from the SLWE distribution, i.e., $\mathbf{B} = \mathbf{AS} + \mathbf{E}$, or a uniformly random matrix. $\mathcal{B}$ directly forwards the challenge matrices $\mathbf{A}, \mathbf{B}$ as the public key of Loong-PKE to $\mathcal{A}$. When $\mathcal{A}$ submits two plaintexts $\mathbf{m}_0, \mathbf{m}_1$, $\mathcal{B}$ uniformly samples a bit $\beta \in \{0, 1\}$. $\mathcal{B}$ leverages the algebraic structure of the SLWE challenge again to generate the ciphertext $(\mathbf{U}, \mathbf{V})$. If $\mathbf{B}$ is a genuine SLWE sample, the distribution of ciphertext $(\mathbf{U}, \mathbf{V})$ is identical to that output by the honest Loong-PKE encryption of $\mathbf{m}_\beta$. If $\mathbf{B}$ is uniformly random, the ciphertext is statistically independent of the plaintext $\mathbf{m}_\beta$. Thus, the advantage of $\mathcal{A}$ in guessing β directly translates to the advantage of $\mathcal{B}$ in distinguishing decision SLWE samples, namely

$$Adv_{IND-CPA}^{Loong-PKE}(\mathcal{A}) \leq 2 \cdot Adv_{SLWE}(\mathcal{B}).$$

□

By using the Fujisaki-Okamoto transformation, we convert the IND-CPA security LoongPKE encryption scheme into the LoongKEM key encapsulation mechanism. We then prove that LoongKEM achieves tight IND-CCA security in the classical random oracle model, and non-tight IND-CCA security in the quantum random oracle model. Following the results in [15], we have the following theorems.

Theorem 2 (Theorem 3.1 and 3.2 in [15]). *For any adversary $\mathcal{B}$ making at most q_D decryption failure queries, q_G queries to random oracle G , and q_H queries to random oracle H , there exists an adversary $\mathcal{A}$ such that*

$$Adv_{LoongKEM}^{IND-CCA}(\mathcal{B}) \leq q_H \cdot \delta + \frac{q_H + 2q_G + 1}{|\mathcal{M}|} + 3 \cdot Adv_{LoongPKE}^{IND-CPA}(\mathcal{A}),$$

where $\mathcal{A}$ runs in essentially the same runtime as $\mathcal{B}$.

Theorem 3 (Theorem 4.4 and 4.6 in [15]). *For any quantum adversary $\mathcal{B}$ making at most q_D classical decryption failure queries, q_G quantum queries to*

random oracle G , q_H quantum queries to random oracle H , and $q_{\widehat{H}}$ quantum queries to random oracle $\widehat{H}$, there exists a quantum adversary $\mathcal{A}$ such that

$$Adv_{LoongKEM}^{IND-CCA}(\mathcal{B}) \leq (2q_{\widehat{H}} + q_H) \cdot \sqrt{8 \cdot \delta(q_G + 1)^2 + (1 + 2q_G)} \sqrt{Adv_{LoongPKE}^{IND-CPA}(\mathcal{A})},$$

where $\mathcal{A}$ runs in essentially the same runtime as $\mathcal{B}$.

5.2 Complexity of the BKZ algorithm

Before evaluating the security of the LoongKEM, we first present the computational complexity of the BKZ lattice reduction algorithm. The BKZ algorithm is currently the state-of-the-art algorithm for finding short lattice vectors. As subroutines, the BKZ algorithm employs either enumeration or sieving. In practice, for lattices of large dimension, sieving achieves the best performance. Therefore, in this paper, we assume that the BKZ algorithm uses sieving as its subroutine. Let β be the dimension of the sublattice for sieving. According to [16, 17, 18, 19], on a classical computer, the time complexity of sieving is $2^{0.292\beta + o(\beta)}$ and the space complexity is $2^{0.2075\beta + o(\beta)}$. According to [20, 21, 22, 23], on a quantum computer, the time complexity of sieving is $2^{0.257\beta + o(\beta)}$, with a quantum space complexity of the same order. Since the dominant cost of the BKZ algorithm stems from the sieving subroutine, we take the computational complexity of sieving as that of the BKZ algorithm.

Let d be the dimension of the lattice $\mathcal{L}$. We next determine the value of β . According to [8, 24], the condition for the BKZ algorithm to successfully output the target short vector $\mathbf{s}$ is

$$\sqrt{\beta/d} \cdot \|\mathbf{s}\| < \delta_\beta^{2\beta - d - 1} \cdot \text{Vol}(\mathcal{L})^{1/d}, \quad (1)$$

where δ_β is the root-Hermite factor of the BKZ algorithm. According to [25], for $\beta > 50$,

$$\delta_\beta = \left(\frac{(\pi\beta)^{1/\beta} \beta}{2\pi e} \right)^{1/(2\beta - 2)}.$$

We can then compute the minimal β ($\beta > 50$) satisfying inequality (1), and further evaluate the computational complexity of the BKZ algorithm.

5.3 Practical security evaluation of the LoongKEM

The LoongKEM still retains some algebraic structure. At present, we do not know how to efficiently exploit such algebraic structure to attack the cryptosystem, so we temporarily ignore these structures. Since the SLWE problem can be regarded as a variant of the original LWE problem, algorithms for solving the LWE problem can be used to solve the SLWE problem.

Let the parameters of the LoongKEM be (q, k_1, k_2, N) , and the public key be $(\mathbf{A}, \mathbf{B})$, where $\mathbf{A} \in \mathbb{Z}^{m \times m}$, $\mathbf{B} \in \mathbb{Z}^{m \times N}$. Let $\mathbf{b}$ be the first column of the matrix $\mathbf{B}$, and $m = (k_1 + k_2)N$.

5.3.1 Primal Attack

Construct the lattice

$$\mathcal{L}_q(\mathbf{A}, \mathbf{I}_m, -\mathbf{b}) = \{\mathbf{x} \in \mathbb{Z}^{2m+1} : (\mathbf{A}, \mathbf{I}_m, -\mathbf{b})\mathbf{x} \mod q = \mathbf{0}\}.$$

The dimension of the lattice is $d = 2m + 1$, and its volume is q^m . It is easy to verify that $(\mathbf{s}, \mathbf{e}, 1)$ is a short vector of the lattice. Thus, we can use the BKZ algorithm to find this short vector. The condition for the BKZ algorithm to successfully find this short vector is $\sigma\sqrt{\beta} < \delta_\beta^{2\beta-d-1} \cdot q^{m/d}$, where σ is the standard deviation of the noise vector.

During an actual attack, since the entries of the vectors $\mathbf{s}$ and $\mathbf{e}$ are small, we can guess the values of some entries of $\mathbf{s}$ or $\mathbf{e}$ and then apply the BKZ algorithm. This can slightly reduce the lattice dimension, thereby lowering the complexity of the BKZ algorithm. However, we cannot guess too many entries of $\mathbf{s}$ or $\mathbf{e}$, otherwise the guessing complexity will become prohibitively high. We need to balance the number of guessed entries and the complexity of the BKZ algorithm. This is known as the Hybrid Primal Attack.

5.3.2 Dual Attack

We first construct the dual lattice

$$\mathcal{L}_q(\mathbf{A}) = \{(\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^{2m} : \mathbf{w}^\top = \mathbf{v}^\top \mathbf{A} \mod q\},$$

whose dimension is $d = 2m$, and volume is q^m . Using the BKZ algorithm, we can find a short vector $(\mathbf{v}, \mathbf{w})$ with length approximately $\ell = \delta_\beta^{d-1} q^{1/2}$. We use this short vector to construct a distinguisher to separate LWE samples from random samples. Given a vector $\mathbf{b} \in \mathbb{Z}^m$, compute $z \equiv \mathbf{v}^\top \mathbf{b} \equiv \mathbf{v}^\top \mathbf{A} \mathbf{s} + \mathbf{v}^\top \mathbf{e} \equiv \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e} \mod q$. If $\mathbf{b} = \mathbf{A} \mathbf{s} + \mathbf{e}$, then z follows a Gaussian distribution with standard deviation $\ell\sigma$. If $\mathbf{b}$ is uniformly distributed, then z is also uniformly distributed. The statistical distance between the Gaussian distribution and the uniform distribution is $\epsilon = 4 \exp(-2\pi^2 \tau^2)$, where $\tau = \ell\sigma/q$. Thus, we need to generate $1/\epsilon^2$ short vectors using the BKZ algorithm. The sieving subroutine of BKZ generates $2^{0.2075\beta}$ short vectors. Assuming all these short vectors have length at most ℓ , we need to run the ϵ -distinguisher $R = \max\{1, 1/2^{0.2075\beta\epsilon^2}\}$ times to finally recover the secret key (or an equivalent key) $\mathbf{s}$.

Similar to the primal attack, in practice, since the entries of vectors $\mathbf{s}$ and $\mathbf{e}$ are small, we can guess the values of some entries of $\mathbf{s}$ or $\mathbf{e}$ and then apply the BKZ algorithm to reduce the attack complexity.

5.3.3 Attack Complexity of the LoongKEM

Table 4 presents the computational complexity of the primal and dual attacks against LoongKEM.

Algorithm	Primal Attack		Dual Attack	
	Classical	Quantum	Classical	Quantum
Loong128	149	136	147	134
Loong256	295	269	292	264
Loong384	407	375	398	361
Loong512	535	496	521	478

Table 4: Computational complexity of primal and dual attacks against LoongKEM. The data in the table are the base-2 logarithms of the attack complexities.

6 Advantages and limitations

The LoongKEM is easy to implement and extend. Implementations of the Loong KEM for different security levels differ only in the macro definitions in the header files, while other code can be reused. If the security level needs to be adjusted, it is only necessary to modify the macro definitions in the header files.

Next, we illustrate the advantages and disadvantages of the LoongKEM by comparing it with other post-quantum key encapsulation mechanisms.

Scheme	sk	pk	ct	gen	enc	dec
Loong128	1832	1472	1512	2621	2647	2676
FrodoKEM-640[13]	19888	9616	9720	1385	1862	1747
Kyber512[26]	1632	800	768	123	155	188
NewHope512[27]	1888	928	1120	117	180	206
ntruhs2048509[28]	935	699	699	8906	643	1662
hqc-128[29]	40	2249	4497	180	408	875
mceliece348864[30]	6492	261120	128	58034	44	135

Table 5: Comparison of key encapsulation algorithms with 128-bit security. The sizes of **sk**, **pk** and **ct** are in Bytes; **gen**, **enc** and **dec** are in 10^3 Cycles. All results are the performance of the reference implementation.

Compared with schemes directly based on the LWE problem There are two main approaches to constructing KEM or key exchange protocols directly from the LWE problem. The first method is detailed in [31], which results in large public and secret keys, but relatively small ciphertexts. The other more practical approach is given in [13], namely the FrodoKEM. As shown in Table 5, the public key and ciphertext sizes of FrodoKEM-640 are approximately six times those of the LoongKEM. In terms of security, key encapsulation mechanisms directly based on the LWE problem have no algebraic structure, whereas the LoongKEM has some algebraic structure.

Compared with schemes based on the MLWE problem A typical key encapsulation mechanism based on the MLWE problem is the Kyber [26], which is the U.S. post-quantum key encapsulation standard. In terms of security, the algebraic structure introduced by Kyber512 is $Z_q[X]/X^{256}+1$, while the algebraic structure introduced by Loong128 is $Z_q[X]/X^{12}+1$. Kyber512 has a stronger algebraic structure, whereas Loong128 has a relatively weaker one. Moreover, part of the public-key matrix of Loong128 has no algebraic structure. In terms of performance, as shown in Table 5, the Kyber512 key encapsulation algorithm significantly outperforms the Loong128 key encapsulation algorithm. The Loong key encapsulation scheme serves as a trade-off between the FrodoKEM and Kyber key encapsulation schemes.

Compared with schemes based on the RLWE problem A typical key encapsulation mechanism based on the RLWE problem is the NewHope[27]. In terms of security, the algebraic structure introduced by NewHope512 is $Z_q[X]/X^{512}+1$, which is stronger than that introduced by Loong128. In terms of performance, as shown in Table 5, the NewHope512 KEM significantly outperforms the Loong128 KEM.

Compared with NTRU In [28], the algebraic structure of the ntruhs2048509 key encapsulation scheme is $Z_q[X]/X^{509}-1$ with $q = 2048$. This algebraic structure is stronger than that introduced by Loong128. In terms of performance, as shown in Table 5, the ntruhs2048509 KEM significantly outperforms the Loong128 KEM.

Compared with schemes based on coding problems Typical key encapsulation mechanisms based on coding problems are the Classic McEliece [30] KEM and the HQC [29] KEM. As shown in Table 5, the total combined length of public key and ciphertext of these two algorithms is longer than that of Loong128. Their public and secret key generation is slower than Loong128, while their key encapsulation and decapsulation are faster than Loong128.

In summary, in practical applications, cryptographic schemes based on the RLWE, MLWE, and NTRU problems all have strong algebraic structures. Cryptographic schemes directly based on the LWE problem suffer from overly long public keys and ciphertexts. If you have concerns about the security of cryptographic schemes with strong algebraic structures and do not wish to use excessively long public keys and ciphertexts, the LoongKEM serves as an excellent trade-off solution.

References

- [1] Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.

- [2] Jeffrey Hoffstein, Jill Pipher, and Joseph H Silverman. Ntru: A ring-based public key cryptosystem. In *International algorithmic number theory symposium*, pages 267–288. Springer, 1998.
- [3] Miklós Ajtai and Cynthia Dwork. A public-key cryptosystem with worst-case/average-case equivalence. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 284–293, 1997.
- [4] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Symposium on the Theory of Computing*, 2005.
- [5] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 1–23. Springer, 2010.
- [6] Jintai Ding, Xiang Xie, and Xiaodong Lin. A simple provably secure key exchange scheme based on the learning with errors problem. *Cryptology ePrint Archive*, 2012.
- [7] Chris Peikert. Lattice cryptography for the internet. In *International workshop on post-quantum cryptography*, pages 197–219. Springer, 2014.
- [8] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. *Cryptology ePrint Archive*, 2015.
- [9] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [10] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroSecP)*, pages 353–367. IEEE, 2018.
- [11] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, Ramesh Karri, Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, et al. Crystals-dilithium. *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30, 2021.
- [12] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
- [13] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *Cryptology ePrint Archive*, 2025.

- [14] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [15] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing.
- [16] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *Annual Cryptology Conference*, pages 3–22. Springer, 2015.
- [17] Anja Becker and Thijs Laarhoven. Efficient (ideal) lattice sieving using cross-polytope lsh. In *International Conference on Cryptology in Africa*, pages 3–23. Springer, 2016.
- [18] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pages 10–24. SIAM, 2016.
- [19] Martin R Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W Postlethwaite, and Marc Stevens. The general sieve kernel and new records in lattice reduction. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 717–746. Springer, 2019.
- [20] Thijs Laarhoven, Michele Mosca, and Van De Pol Joop. Finding shortest lattice vectors faster using quantum search. *Designs, Codes and Cryptography*, 77(2):375–400, 2015.
- [21] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 63–91, 2021.
- [22] Max Heiser. Improved quantum hypercone locality sensitive filtering in lattice sieving. *Cryptology ePrint Archive*, 2021.
- [23] Divesh Aggarwal, Jianwei Li, Phong Q Nguyen, and Noah Stephens-Davidowitz. Slide reduction, revisited-filling the gaps in svp approximation. In *Annual International Cryptology Conference*, pages 274–295. Springer, 2020.
- [24] Tsuyoshi Takagi and Thomas Peyrin. Revisiting the expected cost of solving usvp and applications to lwe. *Springer, Cham*, 10.1007/978-3-319-70694-8(Chapter 11):297–322, 2017.

- [25] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. *DBLP*, 2011.
- [26] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. 2021.
- [27] Erdem Alkim, Roberto Avanzi, Joppe Bos, Léo Ducas, Antonio De La Piedra, Peter Schwabe Thomas Pöppelmann, and Douglas Stebila. Newhope. *Submission to round, 1*, 2017.
- [28] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M Schanck, Peter Schwabe, William Whyte, and Zhenfei Zhang. Algorithm specifications and supporting documentation. *Brown University and Onboard security company, Wilmington USA*, 2019.
- [29] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [30] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [31] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In *Annual international cryptology conference*, pages 554–571. Springer, 2008.